Verifiable Secret Sharing

Limitations, so far

- **Everybody must be honest!**
  - *Dealer, Parties*
  - *Honest-but-curious model*

- **No way to check that protocol rules are followed**
  - *i.e., detect/block cheaters*

- **Need for secret sharing protocols where:**
  - *Party can verify whether dealed share is consistent*
    - Detect malicious dealer
  - *Parties can verify whether revealed share is consistent*
    - Detect cheating parties

Verifiable Secret Sharing

- **VSS**
  - *First (non interactive) scheme proposed in 1987*
    - Feldman VSS scheme
    - Several improvements since then

- **Frequently used as basis for distributed schemes**
  - *More later on this ☺*

- **Basic approach presented next**
  - *Special case of Feldman VSS (using discrete logs)*
  - *Security limitations*

Feldman scheme: dealer

➔Start with an ordinary Shamir scheme

⇨ *Generate random p(x) with degree (t-1), with P(0)=s*

$$p(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-2} x^{t-2} + a_{t-1} x^{t-1}$$

⇨ *Distribute one share to each of the n parties*

$$(x_i, y_i) \quad y_i = p(x_i)$$

In practice, it is convenient to use x_i=i

➔Further step

⇨ *For each coefficient of the polynomial as well as secret, further broadcast*

$$c_0 = g^s; c_1 = g^{a_1}; \cdots c_{t-1} = g^{a_{t-1}} \mod p$$

⇨ *These are special type of "***commitments***"*

➔Discrete log: hard to compute exponent

What is a commitment?

COMMIT phase



a

COMM = Commit(a)





Hiding:

Receiver should NOT learn anything about "a"

REVEAL phase



CANNOT cheat
"I sent a' ≠a"

OPEN= verification key





Binding:

Commitment CAN ONLY open to the value "a", no way for sender to cheat

Verification algorithm:
VERIF(COMM,OPEN,a) = YES/NO

$c=g^x$ is a commitment

➔**(computationally) Hiding:**

⇨*Given $c=g^x$ mod p, computationally bounded receiver cannot know x*

➔**(perfectly) Binding:**

⇨*Sender cannot find any other x' such that $g^{x'}=c$*

➔Note: x' cannot be greater than group order

➔otherwise trivial (why? ☺)

Feldman scheme: verifier

➔ **Party i receives share (x_i,y_i)**

- ⇨ *Other parties can do this, when party i reveals its share*
- ⇨ *Check that party i is honest*

➔ **Verifies that share is valid (hence dealer honest)**

- ⇨ *by computing (mod p)*

$$c_0 \cdot c_1^{x_i} \cdot c_2^{x_i^2} \cdot c_3^{x_i^3} \cdot \ldots \cdot c_{t-1}^{x_i^{t-1}} =$$

$$= \left(g^s\right) \cdot \left(g^{a_1}\right)^{x_i} \cdot \left(g^{a_2}\right)^{x_i^2} \cdot \left(g^{a_3}\right)^{x_i^3} \cdot \ldots \cdot \left(g^{a_{t-1}}\right)^{x_i^{t-1}} =$$

$$= g^s \cdot g^{a_1 x_i} \cdot g^{a_2 x_i^2} \cdot g^{a_3 x_i^3} \cdot \ldots \cdot g^{a_{t-1} x_i^{t-1}} =$$

$$= g^{s + a_1 x_i + a_2 x_i^2 + a_3 x_i^3 + \ldots + a_{t-1} x_i^{t-1}} = g^{p(x_i)}$$

➔ **And checks that this is equal to** g^{y_i}

➔ Where y_i is the party's share

Caveats

- **See exercise for**
  - *Proper moduli (p,q)*
  - *C0 may leak info about secret (Legendre Symbol)*
- **Aftermath:**
  - *use p=2q+1*
  - *Use subgroup order q*

An annoying issue…

- **Shamir Scheme is unconditionally secure**
- **But Feldman's VSS is "only" computationally secure**

- **What does this practically means?**
  - *$c_0 = g^s$ leaks information about secret s!!*
  - *If s is small (e.g. in [1…1000]) trivial "dictionary" attack*
    - For x:1…1000 repeat g^x until $g^x = c_0$ → s known!

- **Can we have perfectly hiding scheme?**

Pedersen commitment

➔**Feldman approach works because:**

⇨*g^a mod p is a Commit(a)*

⇨*And it is homomorphic:*

➔Commit(a+b) = Commit(a) · Commit(b)

➔Indeed: $g^{a+b} = g^a \cdot g^b \bmod p$

➔**We need a perfectly hiding commitment scheme which is homomorphic…**

⇨*Found by Pedersen, 1991 - given g and h (public):*

⇨*Commit(a; r) = $g^a \cdot h^r$ mod p*

$$Commit(a+b; r_a + r_b) = g^{a+b} h^{r_a + r_b} =$$

$$= g^a h^{r_a} \cdot g^b h^{r_b} = Commit(a, r_a) \cdot Commit(b, r_b)$$

Pedersen commitment

➔ **(perfectly) Hiding:**

⇨ *$c=g^a h^r \bmod p$ may be a commitment for* **any possible value***!*

→Formally, for any a'≠a, we can find unique r' such that

$$commit(a';r')=g^{a'}h^{r'} = g^a h^r = commit(a;r)$$

➔ **(computationally) Binding:**

⇨ *Sender should NOT be able to find such an a' (and related r')!*

⇨ *Which is the case if sender does* **NOT** *know* $\log_g h$

→a trapdoor committment scheme!

➔ **What about a perfectly hiding + perfectly binding committment?**

⇨ *Impossible, you may have either one or another,* **not both**

Proof (base of)

let $h = g^w$ i.e. $w = \log_g h$

we know a, r, we are given a', and we look for r' s.t.

$$g^a h^r = g^{a'} h^{r'} \Rightarrow g^a g^{wr} = g^{a'} g^{wr'} \Rightarrow$$

$$\Rightarrow g^{a+wr} = g^{a'+wr'} \Rightarrow$$

$$\Rightarrow a + wr = a' + wr' \quad \mathrm{mod}\, q \text{ (group order, primes.t. } 2q + 1 = p) \Rightarrow$$

$$\Rightarrow r' = w^{-1}(a - a' + wr)$$

i.e., given w, we can compute ANY "colliding" committment, hence we would not commit to anything!! (no binding)

Pedersen VSS - dealer

➔**Generate TWO random polynomials**

$$f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-2} x^{t-2} + a_{t-1} x^{t-1}$$

$$f'(x) = r + b_1 x + b_2 x^2 + \cdots + b_{t-2} x^{t-2} + b_{t-1} x^{t-1}$$

➔**Give party i "double" share (x_i,y_i,z_i)**

$$y_i = f(x_i) = s + a_1 x_i + a_2 x_i^2 + \cdots + a_{t-1} x_i^{t-1}$$

$$z_i = f'(x_i) = r + b_1 x_i + b_2 x_i^2 + \cdots + b_{t-1} x_i^{t-1}$$

➔**Publish commitments**

$$c_0 = g^s h^r$$

$$c_1 = g^{a_1} h^{b_1}$$

...

$$c_{t-1} = g^{a_{t-1}} h^{b_{t-1}}$$

Pedersen VSS - verifier

➔**Party i receives share (x_i,y_i,z_i)**

➔**Verifies (mod p) that:**

$$c_0 \cdot c_1^{x_i} \cdot c_2^{x_i^2} \cdot \ldots \cdot c_{t-1}^{x_i^{t-1}} =$$

$$= \left(g^s h^r\right) \cdot \left(g^{a_1} h^{b_1}\right)^{x_i} \cdot \left(g^{a_2} h^{b_2}\right)^{x_i^2} \cdot \ldots \cdot \left(g^{a_{t-1}} h^{b_{t-1}}\right)^{x_i^{t-1}} =$$

$$= g^s \cdot g^{a_1 x_i} \cdot g^{a_2 x_i^2} \cdot \ldots \cdot g^{a_{t-1} x_i^{t-1}} \cdot h^r \cdot h^{b_1 x_i} \cdot h^{b_2 x_i^2} \cdot \ldots \cdot h^{b_{t-1} x_i^{t-1}} =$$

$$= g^{s + a_1 x_i + a_2 x_i^2 + \ldots + a_{t-1} x_i^{t-1}} \cdot h^{r + b_1 x_i + b_2 x_i^2 + \ldots + b_{t-1} x_i^{t-1}} =$$

$$= g^{y_i} h^{z_i}$$

Still one problem left…

➔**Who "randomly" generates h?**

- ⇨*Dealer?*
  - →Could cheat by generating w and then $h=g^w$
- ⇨*Trusted third party?*
- ⇨*Parties themselves?*

➔**Can we generate $h=g^w$ such that NOBODY knows w?**

- ⇨*Yes, More later on this!*

Distributed Key Generation

Question

➔ **Dlog-based cryptosystems**

- ⇨ **Private key: x; Public key: g^x**
- ⇨ *Many important cases: El Gamal, Identity Based Encryption, …*

➔ **Can we generate (Pub_K, $Priv_K$) pair s.t. all know Pub_K but NOBODY knows $Priv_K$?**

➔ **Useful when:**

- ⇨ *$Priv_K$ to be revealed later on, when needed*
  - →E.g. encrypt message than noone can open, unless special conditions occur
- ⇨ *$Priv_K$ NEVER revealed, but functions of it are needed*
  - →This is extremely important in IBE (more later)

➔ **Other scenarios**

- ⇨ *e.g. generate $h=g^w$ in pedersen VSS*
- ⇨ *And many and many other creative constructions*

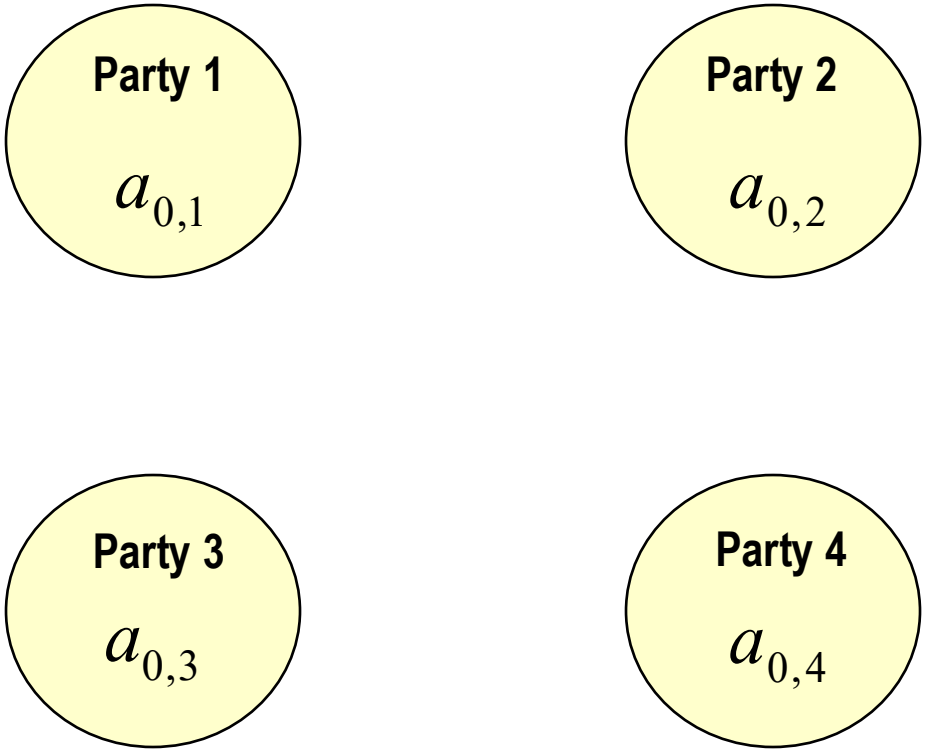
Answer: yes!

- **Schemes called DKG**
  - *Distributed Key Generation*
  - *Fully distributed, NO authority or TTP*

- **Basic one: Pedersen DKG**
  - *1991*
    - "simple" application of what we saw so far
  - *improved and made more secure when cheating parties*
    - Gennaro++ (various works from 1999 to 2007)
  - *We stick to basic approach*
    - Limitations are quite subtle even with cheating parties
    - Pedersen DKG still OK in several practical applications

Best seen by example (3,4)

Step 1: each party independently generates random secret $a_{0,i}$

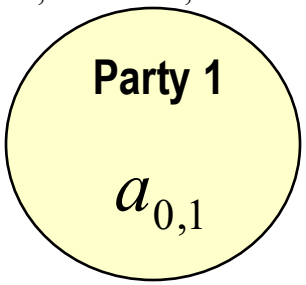


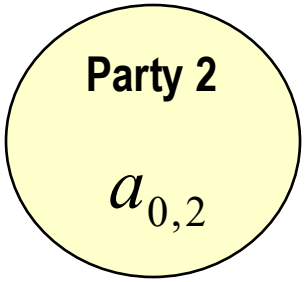
Best seen by example (3,4)

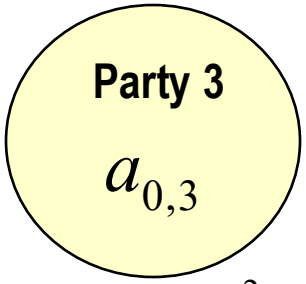
Step 2: generates random polynomial of degree (t-1) with known term $a_{0,i}$

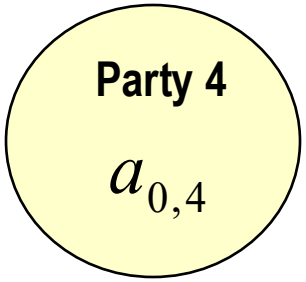
$$p_1(x) = a_{0,1} + a_{1,1}x + a_{2,1}x^2$$

$$p_2(x) = a_{0,2} + a_{1,2}x + a_{2,2}x^2$$







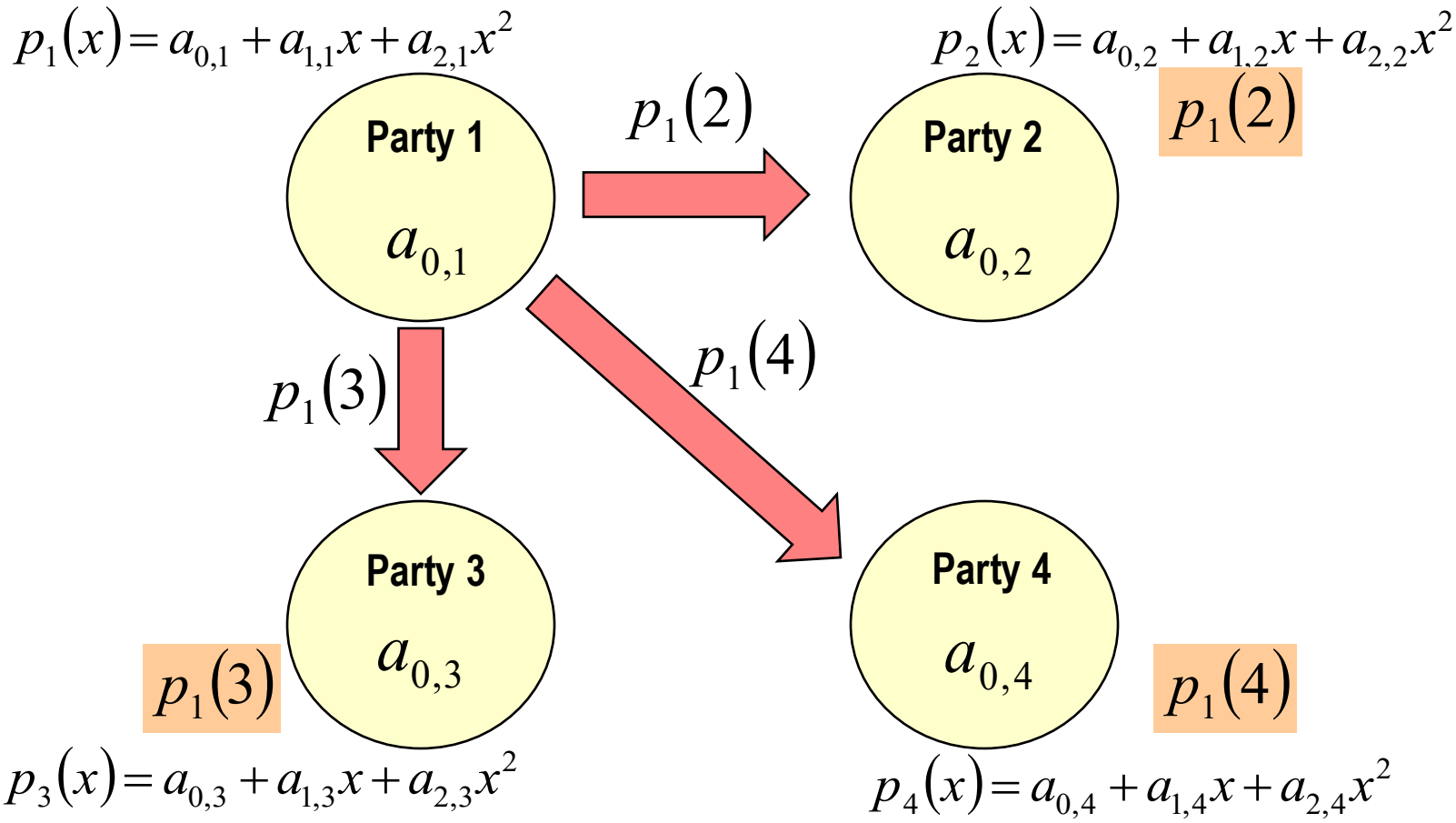


$$p_3(x) = a_{0,3} + a_{1,3}x + a_{2,3}x^2$$

$$p_4(x) = a_{0,4} + a_{1,4}x + a_{2,4}x^2$$

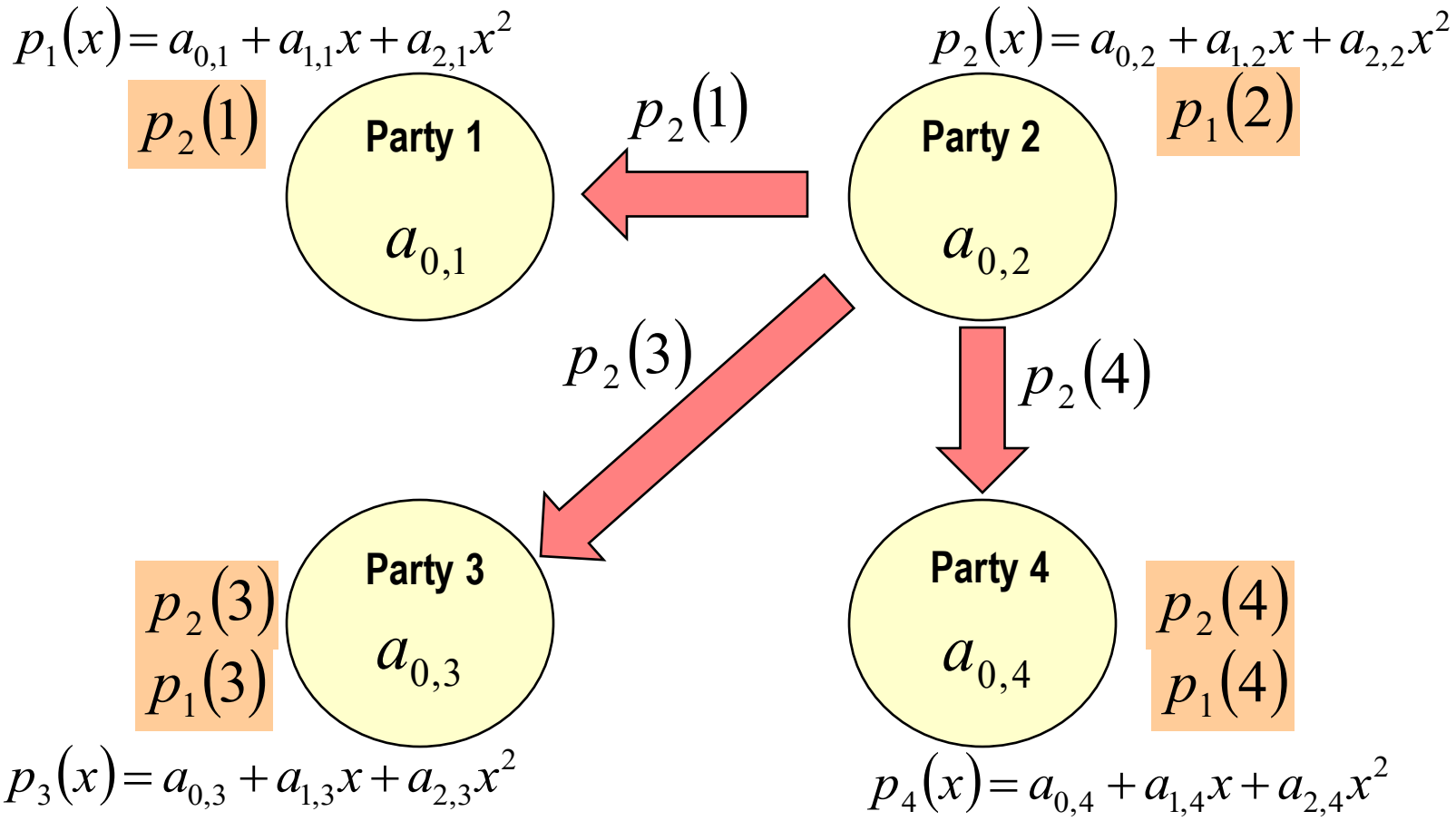
Best seen by example (3,4)

Step 3: securely exchanges share of own poly with every other party



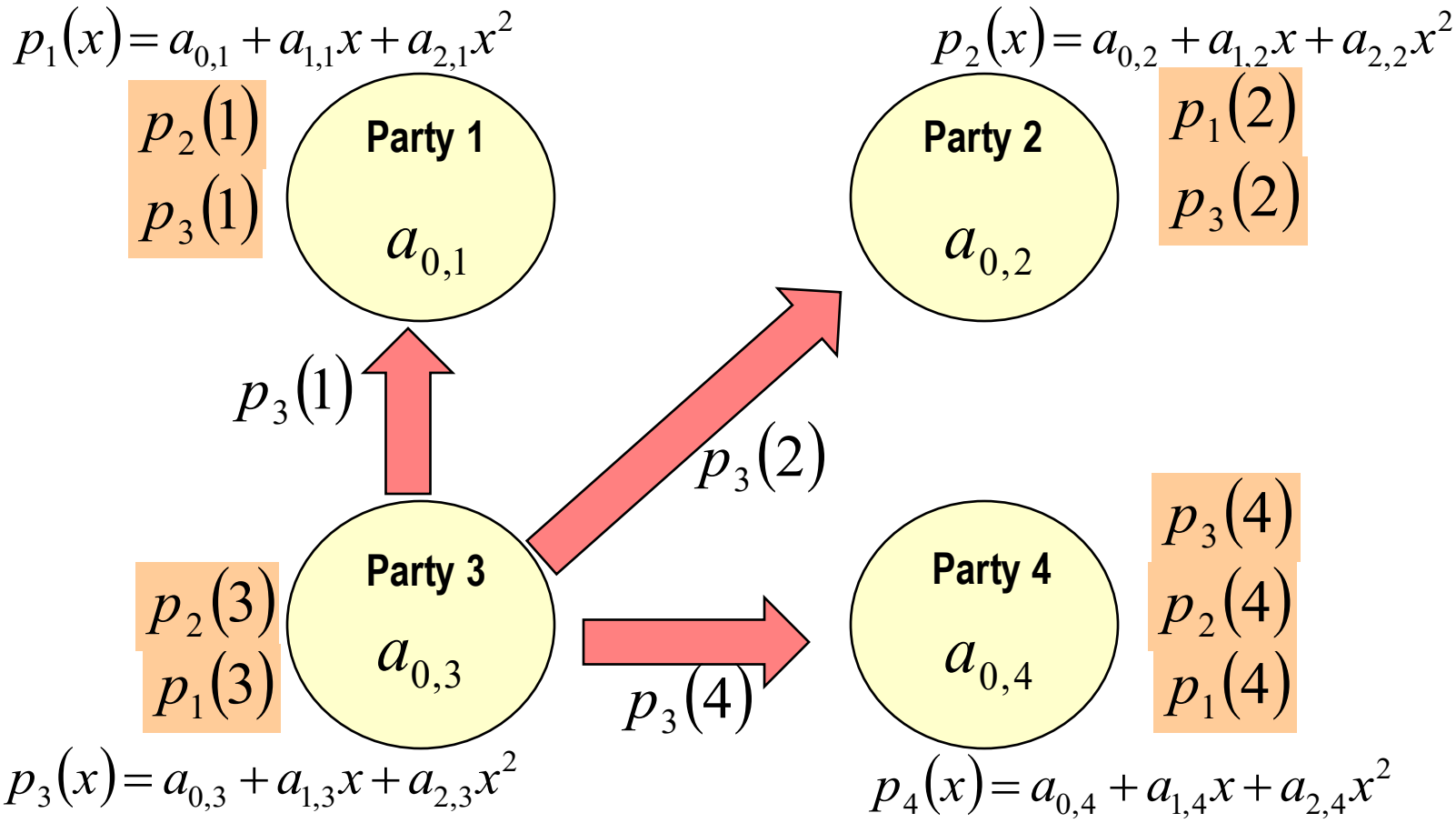
Best seen by example (3,4)

Step 3: securely exchanges share of own poly with every other party



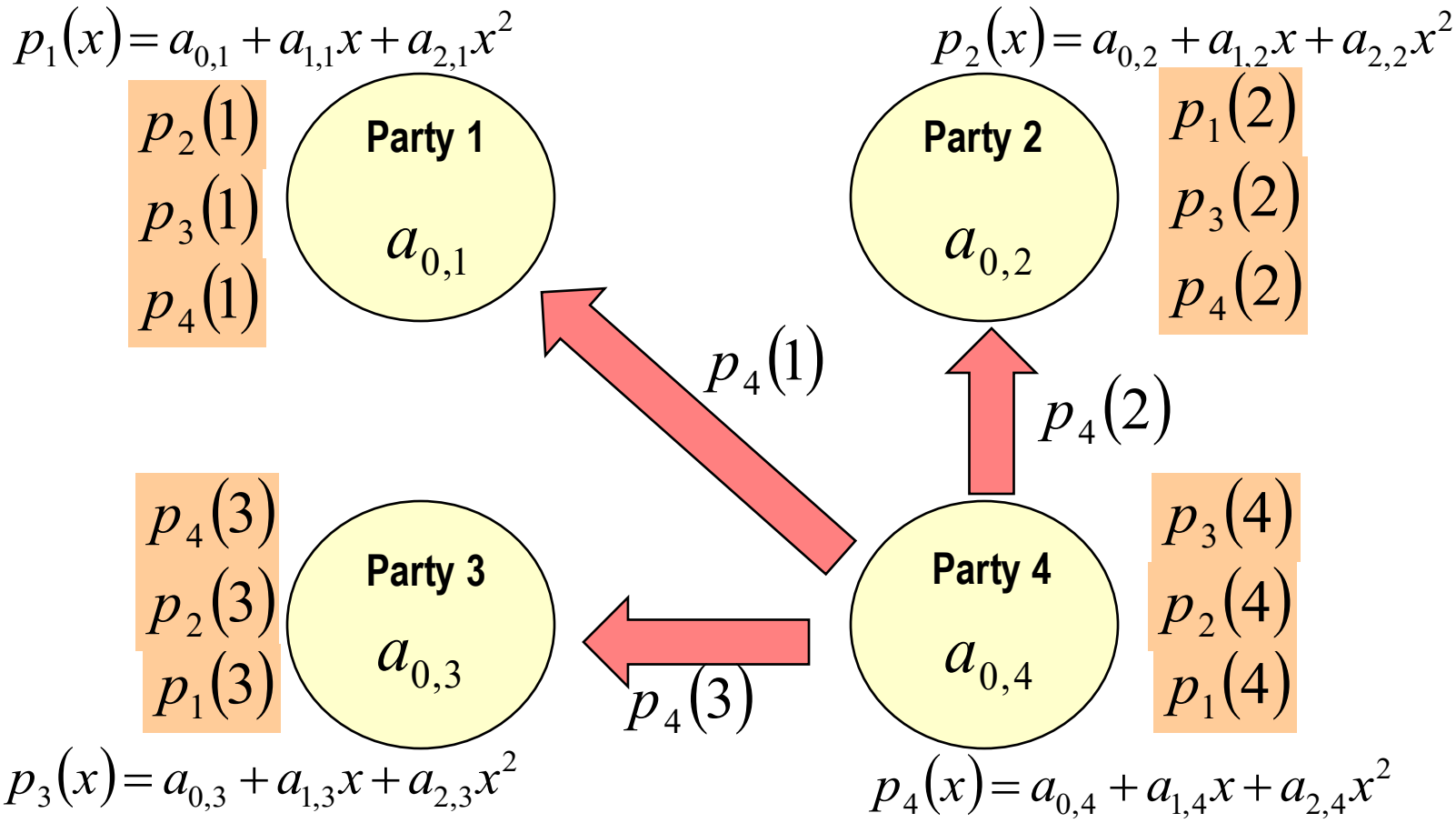
Best seen by example (3,4)

Step 3: securely exchanges share of own poly with every other party



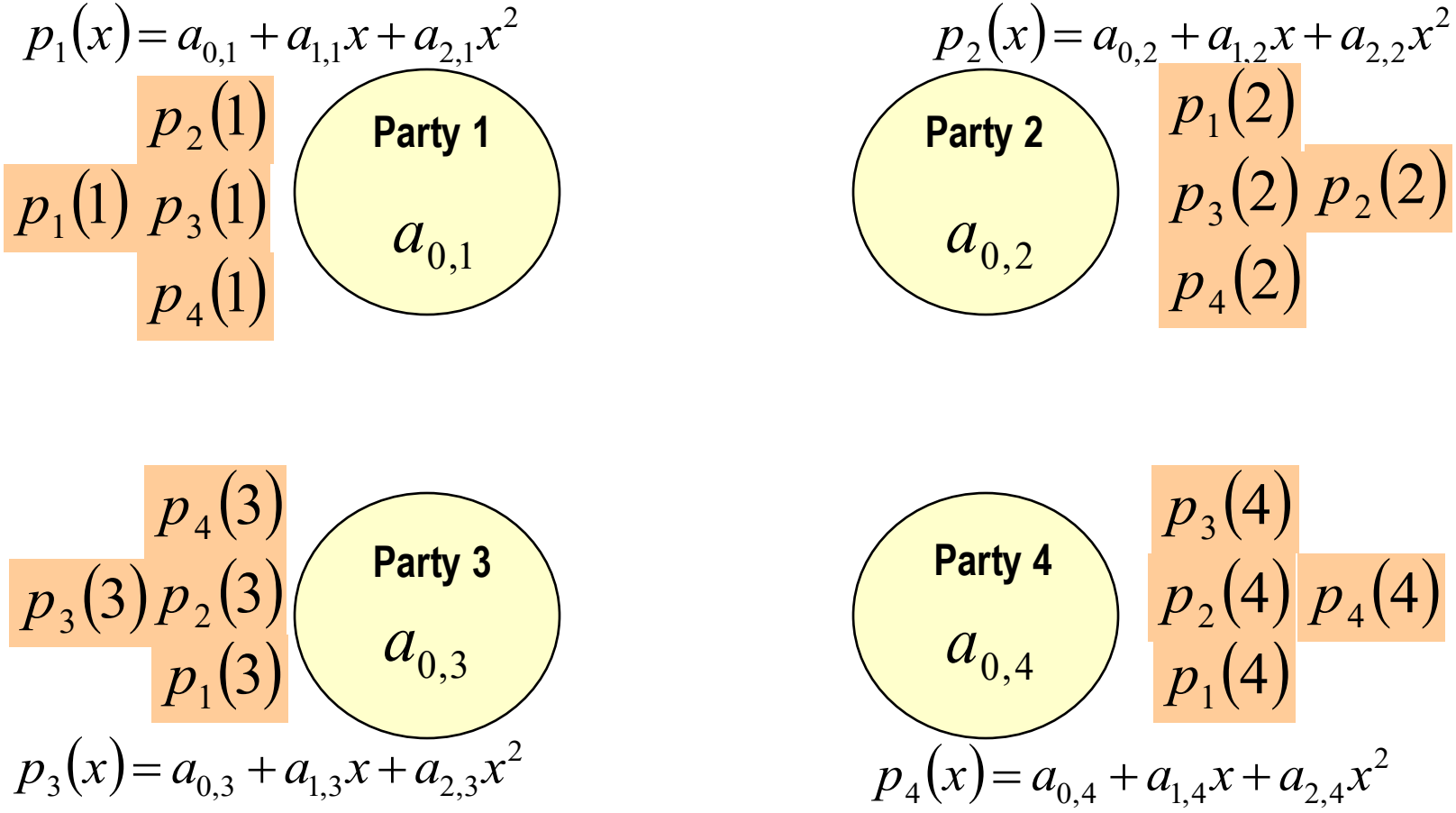
Best seen by example (3,4)

Step 3: securely exchanges share of own poly with every other party



Best seen by example (3,4)

Step 4: each party computes "self" share



Best seen by example (3,4)

Step 5: each party computes "full" share (and keeps it)

$$y_1 = p_1(1) + p_2(1) + p_3(1) + p_4(1)$$

$$p_1(x) = a_{0,1} + a_{1,1}x + a_{2,1}x^2$$

Party 1

$a_{0,1}$

$$y_2 = p_1(2) + p_2(2) + p_3(2) + p_4(2)$$

$$p_2(x) = a_{0,2} + a_{1,2}x + a_{2,2}x^2$$

Party 2

$a_{0,2}$

Party 3

$a_{0,3}$

$$p_3(x) = a_{0,3} + a_{1,3}x + a_{2,3}x^2$$

$$y_3 = p_1(3) + p_2(3) + p_3(3) + p_4(3)$$

Party 4

$a_{0,4}$

$$p_4(x) = a_{0,4} + a_{1,4}x + a_{2,4}x^2$$

$$y_4 = p_1(4) + p_2(4) + p_3(4) + p_4(4)$$

Best seen by example (3,4)

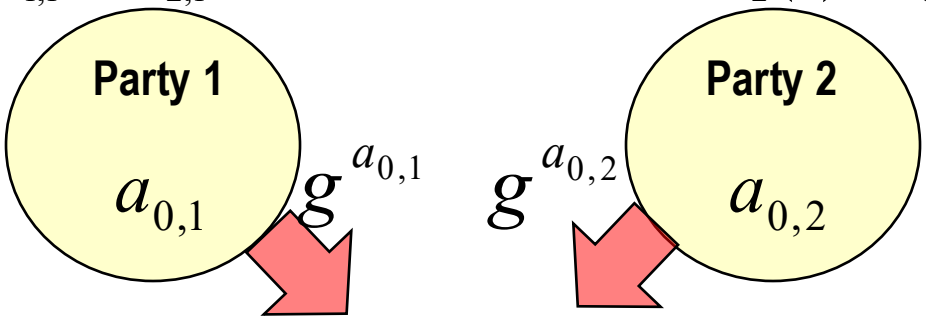
Step 6: …and broadcasts committment for $a_{0,i}$ (if all commitments, full Feldman VSS scheme)

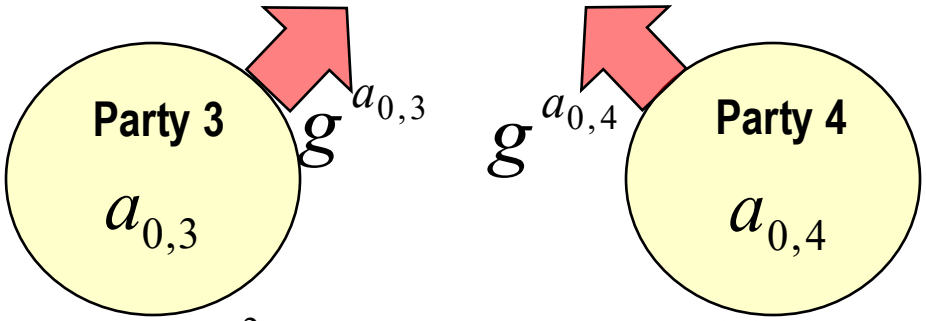
$$y_1 = p_1(1) + p_2(1) + p_3(1) + p_4(1)$$

$$p_1(x) = a_{0,1} + a_{1,1}x + a_{2,1}x^2$$

$$y_2 = p_1(2) + p_2(2) + p_3(2) + p_4(2)$$

$$p_2(x) = a_{0,2} + a_{1,2}x + a_{2,2}x^2$$





$$p_3(x) = a_{0,3} + a_{1,3}x + a_{2,3}x^2$$

$$y_3 = p_1(3) + p_2(3) + p_3(3) + p_4(3)$$

$$p_4(x) = a_{0,4} + a_{1,4}x + a_{2,4}x^2$$

$$y_4 = p_1(4) + p_2(4) + p_3(4) + p_4(4)$$

Best seen by example (3,4)

Wrap up: what we have accomplished so far?

We have distributely built a random polynomial P(x) where a random S is unknown to ALL…

$$\begin{aligned} p_1(x) &= a_{0,1} + a_{1,1}x + a_{2,1}x^2 \\ p_2(x) &= a_{0,2} + a_{1,2}x + a_{2,2}x^2 \\ p_3(x) &= a_{0,3} + a_{1,3}x + a_{2,3}x^2 \\ p_4(x) &= a_{0,4} + a_{1,4}x + a_{2,4}x^2 \end{aligned}$$

$$\sum \quad P(x) = s + \alpha_1 x + \alpha_2 x^2$$

Being:

$$s = \sum_{i=1}^{4} a_{0,i}$$

$$\alpha_1 = \sum_{i=1}^{4} a_{1,i}$$

$$\alpha_2 = \sum_{i=1}^{4} a_{2,i}$$

… but where each party has a valid SHARE of such random secret S…

$$y_i = p_1(i) + p_2(i) + p_3(i) + p_4(i) = P(i)$$

… and (our target!) where each party can compute g^s using the broadcast committments!

$$g^{a_{0,1}} \cdot g^{a_{0,2}} \cdot g^{a_{0,3}} \cdot g^{a_{0,4}} = g^{a_{0,1}+a_{0,2}+a_{0,3}+a_{0,4}} = g^s$$